



RETRIEVAL-AUGMENTED GENERATION

# Chunking Techniques for RAG Systems

*Strategies, embeddings, and vector stores for production-grade retrieval*

# What We'll Cover



## Chunking Strategies

Seven techniques, from fixed-size to agentic splitting



## Text Representation

Dense, sparse, and graph-based encodings



## Vector Databases

FAISS, Pinecone, Weaviate, and ChromaDB



## Implementation Examples

Working code for every technique



## Comparison & Best Practices

Choosing the right approach for your data

# 7

## CHUNKING TECHNIQUES

*from simple fixed-size splits to LLM-guided agentic chunking — each with trade-offs in speed, quality, and cost.*

# What is RAG?

*Retrieval-Augmented Generation*



## **Combines retrieval with generation**

Pulls relevant context from an external knowledge base before the LLM generates a response



## **Reduces hallucinations**

Grounds responses in factual, retrievable data rather than model memory alone



## **Extends LLM knowledge**

Enables access to up-to-date or domain-specific information beyond training data



## **Why Chunking Matters**

*Chunks are the atomic units of retrieval in RAG systems.*

How you split documents directly determines what the model can ever retrieve — too coarse loses precision, too fine loses context.



PART 1

# Chunking Strategies

*Implementation examples & best practices across seven techniques*

# 1 Fixed-Size Chunking

- **Characters / tokens**

Cuts text exactly after a defined number of characters (e.g., 100) or tokens

- **Overlap**

A `chunk_overlap` parameter shares content between chunks so context isn't lost at the cut

- **No structure awareness**

Blindly splits at the character count, often mid-word or mid-sentence

## PROS

Simple, fast, predictable

## CONS

May break semantic boundaries

**USE CASE** Quick prototyping, uniform processing needs

## IMPLEMENTATION

```
def fixed_size_chunk(text, size=512):
    chunks = []
    for i in range(0, len(text), size):
        chunk = text[i:i+size]
        chunks.append(chunk)
    return chunks
```

# Usage

```
text = "Long document..."
chunks = fixed_size_chunk(text, 512)
```

## 2 Recursive Chunking

- **Hierarchical splitting**

Splits using a cascade of separators — paragraphs, then sentences, then characters

- **Structure-preserving**

Falls back gracefully when a separator isn't found, keeping natural text boundaries

### PROS

Preserves structure  
better than fixed-size

### CONS

More complex, slower

**USE CASE** General-purpose text, maintaining readability

### IMPLEMENTATION

```
from langchain.text_splitter import \
    RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=['\n\n', '\n', '. ', ' ']
)
chunks = splitter.split_text(text)
```

# 3 Semantic Chunking (Basic)

## IMPLEMENTATION

- Description**

Groups sentences based on embedding similarity rather than fixed length

### PROS

Semantically coherent chunks

### CONS

Computationally expensive

**USE CASE** High-quality retrieval, topic-based segmentation

Sentence 1: Cats are wonderful pets.  
Sentence 2: They enjoy sleeping.  
Sentence 3: Dogs are also popular pets.  
Sentence 4: The stock market rose today.  
Sentence 5: Investors were optimistic.

$S1 \leftrightarrow S2 = 0.95$   
 $S2 \leftrightarrow S3 = 0.82$   
 $S3 \leftrightarrow S4 = 0.18$   
 $S4 \leftrightarrow S5 = 0.93$

Cosine similarity ranges from -1 to 1:

1 → identical meaning  
0 → unrelated  
-1 → opposite meanings

In practice, sentence similarities are usually between 0 and 1.

```
from langchain_experimental.text_splitter \
    import SemanticChunker
from langchain_openai import OpenAIEmbeddings

chunker = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type='percentile'
)
chunks = chunker.create_documents([text])
```

# 4 Semantic Chunking (Adaptive)

## IMPLEMENTATION

- **Description**

Dynamically adjusts chunk boundaries using similarity thresholds between sentence embeddings

### PROS

Better handles varying content density

### CONS

Variable chunk sizes, complex tuning

**USE CASE** Streaming text, variable-density documents

```
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(sentences)

sims = [util.cos_sim(embeddings[i-1],
                     embeddings[i]).item()
         for i in range(1, len(sentences))]
threshold = np.percentile(sims, 25)

# Build chunks where similarity >= threshold
chunks, current = [], [sentences[0]]
for i in range(1, len(sentences)):
    if sims[i-1] < threshold:
        chunks.append(' '.join(current))
        current = [sentences[i]]
    else:
        current.append(sentences[i])
if current:
    chunks.append(' '.join(current))
```

# 5 Document-Based Chunking

## IMPLEMENTATION

- **Description**

Splits by document structure such as markdown headers and sections

- **Hierarchy-aware**

Keeps each chunk tied to the section it belongs to, preserving navigational context

### PROS

Preserves document hierarchy and context

### CONS

Requires structured documents

**USE CASE** Technical docs, wikis, structured content

```
from langchain.text_splitter import \
    MarkdownHeaderTextSplitter

headers = [
    ('#', 'Header 1'),
    ('##', 'Header 2'),
    ('###', 'Header 3')
]
splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers
)
chunks = splitter.split_text(md_text)
```

# 6 Agentic Chunking (LLM-Guided)

## IMPLEMENTATION

- **Description**

Uses an LLM to intelligently determine where chunk boundaries should fall

- **Context-aware**

Understands meaning the way a human editor would, not just statistical similarity

### PROS

Highest quality, context-aware splitting

### CONS

Expensive, slow, requires API calls

**USE CASE** Premium-quality retrieval where cost is secondary

```
from openai import OpenAI
client = OpenAI()

def agentic_chunk(text):
    prompt = f'''Split this text into
semantically coherent chunks.
Return as JSON list. Text: {text}'''
    response = client.chat.completions.create(
        model='gpt-4',
        messages=[{'role': 'user',
                    'content': prompt}]
    )
    return parse_chunks(response)
```

# 7 Parent-Document Retrieval

## IMPLEMENTATION

- **Description**

Indexes small child chunks for search, but retrieves their larger parent documents for generation

- **Best of both**

Combines precise matching with full surrounding context at answer time

### PROS

Precision in search,  
context in generation

### CONS

Complex indexing,  
requires mapping

**USE CASE** Long documents needing both precision and context

```
from langchain.retrievers import \
    ParentDocumentRetriever
from langchain.storage import InMemoryStore

parent_splitter = RecursiveCharacterTextSplitter(
    chunk_size=2000)
child_splitter = RecursiveCharacterTextSplitter(
    chunk_size=400)

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=InMemoryStore(),
    parent_splitter=parent_splitter,
    child_splitter=child_splitter
)
```

# Chunking Strategy Comparison

| Strategy            | Speed     | Quality   | Cost   | Best For            |
|---------------------|-----------|-----------|--------|---------------------|
| Fixed-Size          | Very Fast | Low       | Free   | Prototyping         |
| Recursive           | Fast      | Medium    | Free   | General use         |
| Semantic (Basic)    | Slow      | High      | Medium | Topic segmentation  |
| Semantic (Adaptive) | Slow      | Very High | Medium | Variable content    |
| Document-Based      | Fast      | High      | Free   | Structured docs     |
| Agentic             | Very Slow | Highest   | High   | Premium quality     |
| Parent-Document     | Medium    | High      | Free   | Precision + context |

# TASK

Compare how different chunking strategies split the same document.

**Instructions:**

1. Use the provided **AI article**.
2. Apply these three chunking strategies:
  1. Fixed-Size Chunking
  2. Recursive Chunking
  3. Semantic Chunking
3. For each strategy, display:
  1. Number of chunks
  2. Average chunk length
  3. Largest and smallest chunk
  4. Execution time
  5. The first two chunks



A

PART 2

# Text Representation

*Encoding chunks for retrieval*

# Why Text Representation Matters



## Dense Embeddings

Semantic similarity in vector space



## Sparse Vectors

Keyword-based retrieval  
(BM25, TF-IDF)



## Knowledge Graphs

Structured entity relationships

# A Dense Embeddings (Semantic)

## IMPLEMENTATION

- **High-dimensional vectors**

Converts text into vectors with 384-1536 dimensions

- **Semantic meaning**

Captures meaning and synonyms, not just exact keyword overlap

- **Common models**

OpenAI, Sentence-BERT, Cohere, and similar embedding models

### PROS

Excellent for semantic search, synonyms

### CONS

Needs a model + compute to encode

**USE CASE** General-purpose semantic search and RAG retrieval

```
from sentence_transformers import \
    SentenceTransformer

# Load model
model = SentenceTransformer(
    'all-MiniLM-L6-v2')

# Encode chunks
chunks = ['chunk 1', 'chunk 2']
embeddings = model.encode(chunks)
# embeddings.shape -> (2, 384)
```

# B Sparse Vectors (Keyword / Lexical)

- **Term-frequency based**

Represents documents as sparse vectors based on term frequency

- **Algorithms**

BM25 and TF-IDF are the standard scoring functions

- **Exact matching**

Excellent for exact keyword matches that embeddings can miss

## PROS

Fast, interpretable, good for exact terms

## CONS

Misses synonyms and paraphrasing

**USE CASE** Keyword search, hybrid retrieval pairing

## IMPLEMENTATION

```
from rank_bm25 import BM25Okapi

# Tokenize chunks
tokenized_chunks = [
    chunk.split() for chunk in chunks]

# Create BM25 index
bm25 = BM25Okapi(tokenized_chunks)

# Search
query = 'machine learning'
scores = bm25.get_scores(query.split())
top_n = scores.argsort()[::-1][-5:]
```

# C Knowledge Graph Representation

- **Entity extraction**

Extracts entities and relationships from text to form a graph

- **Multi-hop reasoning**

Enables relationship queries that span multiple connected facts

- **Common tools**

Neo4j and LangChain's GraphIndexCreator

## PROS

Captures structured relationships

## CONS

Heavier setup, harder to scale

**USE CASE** Multi-hop reasoning, relationship-heavy domains

## IMPLEMENTATION

```
from langchain.indexes import \
    GraphIndexCreator
from langchain.llms import OpenAI

index_creator = GraphIndexCreator(
    llm=OpenAI(temperature=0))
graph = index_creator.from_text(text)

# Query the graph
graph.query(
    'What is the relationship between
    entity A and entity B?')
```

## TEXT REPRESENTATION

# Hybrid Search: Best of Both Worlds

*Combining dense (semantic) and sparse (keyword) retrieval*



Dense vectors capture semantic meaning and synonyms



Sparse vectors ensure exact keyword matches aren't missed



Results are merged using Reciprocal Rank Fusion (RRF)

## SCORE FUSION

```
hybrid_results = alpha * dense_scores + (1 - alpha) * sparse_scores  
# Common alpha values: 0.5 to 0.7
```



PART 3

# Vector Databases

*Storing and retrieving embeddings*

# What is a Vector Database?



## Purpose-built for similarity search

Optimized specifically for vector similarity search at scale



## Stores high-dimensional embeddings

Handles vectors with hundreds to thousands of dimensions



## Fast approximate nearest neighbor

Performs fast ANN search instead of slow exhaustive comparison



## Scales to millions of vectors

Enables efficient retrieval at scale across massive datasets



## Key Features

- Similarity metrics (cosine, euclidean)
- Filtering and metadata support
- Horizontal scaling



# FAISS

Facebook AI Similarity Search

## TYPE

Open-source library (not a database)

### PROS

Extremely fast, GPU support, highly optimized

### CONS

No built-in persistence, limited metadata

**USE CASE** Research, local prototyping

## QUICK START

```
from langchain_community.vectorstores \
    import FAISS
from langchain_huggingface import \
    HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name='all-MiniLM-L6-v2')
vectorstore = FAISS.from_texts(
    chunks, embeddings)

# Retrieve
result = vectorstore.similarity_search(
    'What is machine learning?', k=5)
```



# Pinecone

*Managed Vector Database*

## TYPE

Fully managed cloud service

### PROS

Easy to use, auto-scaling, high availability

### CONS

Cost at scale, vendor lock-in

**USE CASE** Production apps, scalable RAG

## QUICK START

```
import pinecone
from langchain_pinecone import \
    PineconeVectorStore
from langchain_huggingface import \
    HuggingFaceEmbeddings

pc = pinecone.Pinecone(api_key='KEY')
embeddings = HuggingFaceEmbeddings(
    model_name='all-MiniLM-L6-v2')
vectorstore = PineconeVectorStore.from_texts(
    texts=chunks, embedding=embeddings,
    index_name='my-rag-index')
```



# Weaviate

*Open-Source Vector Database*

## TYPE

Open-source, self-hosted or cloud

### PROS

GraphQL API, hybrid search, module ecosystem

### CONS

More complex setup than managed options

**USE CASE** Advanced RAG, knowledge graphs

## QUICK START

```
import weaviate
from langchain_weaviate import \
    WeaviateVectorStore
from langchain_huggingface import \
    HuggingFaceEmbeddings

client = weaviate.connect_to_local()
embeddings = HuggingFaceEmbeddings(
    model_name='all-MiniLM-L6-v2')
vectorstore = WeaviateVectorStore.from_documents(
    client=client, documents=chunks,
    embedding=embeddings,
    index_name='MyRAGIndex')
```



# ChromaDB

*Embeddable Vector Database*

## TYPE

Open-source, embeddable database

### PROS

Simple API, local-first, LangChain integration

### CONS

Limited scaling for very large datasets

**USE CASE** Development, small-to-medium RAG apps

## QUICK START

```
from langchain_chroma import Chroma
from langchain_huggingface import \
    HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name='all-MiniLM-L6-v2')
vectorstore = Chroma.from_texts(
    chunks, embedding=embeddings)

# Retrieve
result = vectorstore.similarity_search(
    'What is machine learning?', k=5)
```

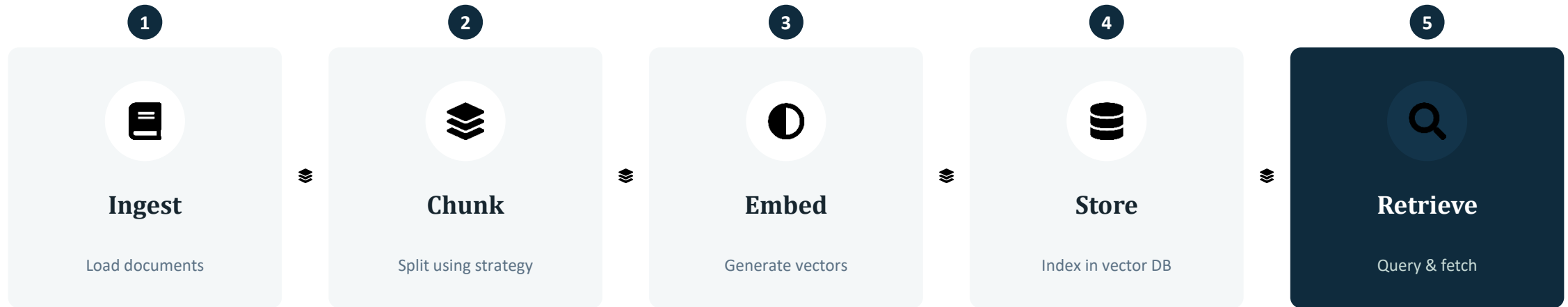
# Vector Database Comparison

| Database | Type     | Deployment  | Best For                 | Pricing       |
|----------|----------|-------------|--------------------------|---------------|
| FAISS    | Library  | Local       | Research, prototyping    | Free          |
| Pinecone | Managed  | Cloud       | Production, scale        | Pay-as-you-go |
| Weaviate | Database | Self/Cloud  | Hybrid search, graphs    | Free/Paid     |
| ChromaDB | Embedded | Local/Cloud | Development, simple apps | Free          |

**KEY CONSIDERATION** Choose based on scale, budget, and deployment preferences. Start simple (FAISS / ChromaDB), scale as needed (Pinecone / Weaviate).

## PUTTING IT TOGETHER

# Complete RAG Pipeline



## EXAMPLE CODE

```
docs = load_documents('data/')
chunks = splitter.split(docs)
embeddings = model.encode(chunks)
vectorstore.add(embeddings, chunks)
results = vectorstore.search(query)
```

```
# 1. Ingest
# 2. Chunk
# 3. Embed
# 4. Store
# 5. Retrieve
```

# Best Practices for RAG Systems



## Chunking

- Test multiple strategies
- Use overlap for context
- Chunk size: 200-1000 tokens



## Embeddings

- Use domain-specific models
- Consider hybrid search
- Cache embeddings



## Retrieval

- Tune top-k parameter
- Use metadata filtering
- Re-rank results



## Evaluation

- Track retrieval accuracy
- Monitor latency
- A/B test strategies

# Common Pitfalls to Avoid



## Too Small Chunks

Loss of semantic meaning, context fragmentation



## Too Large Chunks

Loss of precision, irrelevant context retrieved



## No Overlap

Information lost at chunk boundaries



## Ignoring Metadata

Missing filtering opportunities (date, source, author)



## Single Retrieval Strategy

Not testing hybrid or re-ranking approaches



## Poor Embedding Model

Generic models may underperform on domain-specific text

# Resources & Next Steps



## Documentation

- [LangChain Docs](#)
- [FAISS GitHub](#)
- [Pinecone Learning Center](#)
- [Weaviate Documentation](#)
- [ChromaDB Cookbook](#)



## Next Steps

- Experiment with chunking strategies on your data
- Set up a vector database locally
- Build a simple RAG prototype
- Evaluate retrieval quality
- Optimize for production



# Thank You

---

Eng \ Nancy Ahmed